

RAPPRESENTAZIONE DELLE INFORMAZIONI

Sistemi numerici

Bit: unita' di misura dell'informazione. Puo' assumere valore 0 o 1

Una rappresentazione e' un modo per descrivere un'entita'

Avere a disposizione piu' bit significa piu' precisione nella rappresentazione dei numeri

nybble	4 bit
byte	8 bit
halfword	16 bit
word	32 bit
doubleword	64 bit

Sistema posizionale

Il valore assunto da una cifra varia a seconda della sua posizione nel numero

Formula: $N = \sum_{i=-m}^{n-1} d_i \cdot r^i$
 d = singola cifra
 r = radice o base del sistema
 n = numero di cifre p. intera
 m = numero di cifre p. frazionaria

Sistema decimale: base = 10
 cifre = 0, 1, ..., 9

$$N = d_{n-1} \cdot 10^{n-1} + \dots + d_0 \cdot 10^0 + d_{-1} \cdot 10^{-1} + \dots + d_{-m} \cdot 10^{-m}$$

Sistema binario: base = 2
 cifre = 0, 1

$$N = d_{n-1} \cdot 2^{n-1} + \dots + d_0 \cdot 2^0 + d_{-1} \cdot 2^{-1} + \dots + d_{-m} \cdot 2^{-m}$$

Sistema esadecimale: base = 16
 cifre = 0, 1, ..., 9, a, b, c, d, e, f

$$N = d_{n-1} \cdot 16^{n-1} + \dots + d_0 \cdot 16^0 + d_{-1} \cdot 16^{-1} + \dots + d_{-m} \cdot 16^{-m}$$

↳ vantaggi: compattezza dei numeri

usato in: Internet Protocol (IP), codice ASCII, CSS

Il grande vantaggio del sistema esadecimale e la compattezza dei numeri

La base 16 consente di utilizzare meno cifre rispetto al sistema binario o a quello decimale

↳ un'informazione in 16 cifre binarie puo essere rappresentata in 4 cifre esadecimali

0hex = 0dec = 0oct	0	0	0	0
1hex = 1dec = 1oct	0	0	0	1
2hex = 2dec = 2oct	0	0	1	0
3hex = 3dec = 3oct	0	0	1	1
4hex = 4dec = 4oct	0	1	0	0
5hex = 5dec = 5oct	0	1	0	1
6hex = 6dec = 6oct	0	1	1	0
7hex = 7dec = 7oct	0	1	1	1
8hex = 8dec = 10oct	1	0	0	0
9hex = 9dec = 11oct	1	0	0	1
Ahex = 10dec = 12oct	1	0	1	0
Bhex = 11dec = 13oct	1	0	1	1
Chex = 12dec = 14oct	1	1	0	0
Dhex = 13dec = 15oct	1	1	0	1
Ehex = 14dec = 16oct	1	1	1	0
Fhex = 15dec = 17oct	1	1	1	1

Conversioni tra sistemi numerici

- conversione da qualsiasi base (r) a base 10

$$N_r = d_{n-1} \cdot r^{n-1} + \dots + d_0 \cdot r^0 = M_{10}$$

$$\text{esempio: } 1010_2 = (1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0)_{10} = 10_{10}$$

- conversione da base 10 a qualsiasi base (r)

1. divido il valore numerico N per la base r fino a quando l'ultimo quoziente e' minore della base

2. Prendo l'ultimo quoziente e tutti i esiti delle divisioni e procedo dall'ultimo resto al primo

$$\text{esempio } 12_{10} \rightarrow N_2 \quad \left. \begin{array}{l} 12 / 2 = 6 \text{ resto } 0 \\ 6 / 2 = 3 \text{ resto } 0 \\ 3 / 2 = 1 \text{ resto } 1 \\ 1 / 2 = 0 \text{ resto } 1 \end{array} \right\} \rightarrow 12_{10} = 1100_2$$

Osservazione

La conversione da base 2 a base 8/16 è più semplice di quella da decimale o altre basi

- Per rappresentare le 16 cifre dell'esadecimale occorrono 4 bit binari
- Per rappresentare le 8 cifre dell'ottale occorrono 3 bit binari

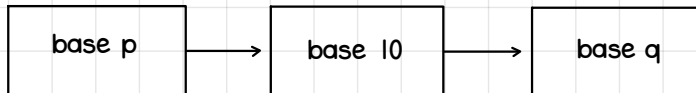
Quindi per convertire da binario a esadecimale/ottale è sufficiente raggruppare le cifre in gruppi di 4/3 a partire da quelle meno significative

Esempio: 0111 - 1111 - 0001 - 1010

7 - 15 - 1 - 10 → 7F1A₁₆

convertito a gruppi di 4 le cifre in decimale e poi in esadecimale

La conversione da qualsiasi base p a qualsiasi base q avviene nel seguente modo:



Rappresentazione dei valori

La rappresentabilità dei valori è legata al numero di cifre

disponibili. Il numero di cifre impiegate nella rappresentazione dei valori numerici è infatti limitato.

Si ha overflow quando si è nell'impossibilità di rappresentare il risultato dell'operazione

$$\text{KILO} = 2^{10} = 1.024$$

$$\text{MEGA} = 2^{20} = 1.048.576$$

$$\text{GIGA} = 2^{30} = 1.073.741.824$$

n = num di bit	2 ⁿ num di valori	2 ⁿ -1 max val. rappresentabile
1	2	1
2	4	3
3	8	7
4	16	15
5	32	31

Sistema binario: operazioni aritmetiche

Somma:

si esegue la somma tra i bit di pari ordine

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ con riporto sul bit di ordine superiore}$$

$$1 + 1 + 1 = 1 \text{ con riporto sul bit di ordine superiore}$$

$$\text{ESEMPIO: } 010011 + 010001 = 100100 \quad (19 + 17 = 36)$$

RIPORTO	1	0	0	1	1	
1° ADDENDO	0	1	0	0	1	1
2° ADDENDO	0	1	0	0	0	1
SOMMA	1	0	0	1	0	0

- la somma è definita su 3 elementi: due addendi e il riporto (carry)
- la somma di 2 unità crea 1 unità di ordine immediatamente superiore (carry)

Sottrazione:

si esegue la differenza tra i bit di pari ordine

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1 \text{ con prestito dal bit di ordine superiore}$$

$$\text{ESEMPIO: } 11101 - 01110 = 01111 \quad (29 - 14 = 15)$$

PRESTITO			2	2	2	
MINUENDO		0	1	0	1	1
SOTTRAENDO		0	1	1	1	0
DIFFERENZA		0	1	1	1	1

- è definita su 3 elementi: minuendo, sottraendo e prestito (borrow)
- ogni volta che si deve svolgere 0-1 occorre chiedere in prestito un'unità dalla cifra superiore che vale 2 unità della cifra di ordine inferiore

Quando si hanno a disposizione un certo numero di bit ma non si riesce a far rientrare il risultato in quel range si ha OVERFLOW

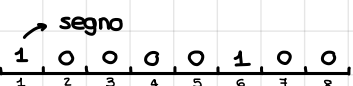
Se invece durante la sottrazione il sottraendo è maggiore del minuendo il risultato è un numero negativo ma nel sistema binario puro non è possibile rappresentare numeri negativi

Modulo e segno

Per ovviare al problema della rappresentazione dei numeri negativi è stato definito il metodo di "modulo e segno"

Supponendo di avere a disposizione 1 Byte (8 bit) per rappresentare numeri positivi e negativi

- i primi 7 bit da destra saranno usati per esprimere il Valore assoluto del numero
- Il bit più a sinistra (MSB) per indicare il segno -> 1 se il numero è negativo
-> 0 se il numero è positivo

ESEMPIO: $-4_{10} =$ 

Con n bit totali si possono rappresentare i numeri dell'intervallo: $[-(2^{n-1}-1), +(2^{n-1}-1)]$

Esistono però due problemi per quanto riguarda la rappresentazione MS

- due diverse rappresentazioni dello 0
utilizzando ad esempio 4 bit totali si ha: $000p = +0$ e $1000 = -0$
- 1 bit viene sempre perso per rappresentare il segno

Complemento a 1 (CA1)

Questo metodo si basa sull'operazione di complemento

Con complemento si intende l'operazione che associa ad un bit il suo opposto

- se il numero da codificare è positivo lo si converte in binario con il metodo classico
- se il numero è negativo si converte il suo modulo e si effettua l'operazione di complemento

ESEMPIO: supponendo di avere 4 bit e di voler rappresentare 3_{10} e -3_{10}

i risultati sarebbero: $3_{10} = 0011_2$ e $-3_{10} = \overline{0011}_2 = 1100$

Anche in questo caso sussiste il problema della doppia rappresentazione dello zero: $0000 (+0)$ $1111 (-0)$

Complemento a 2 (CA2)

Un altro metodo di codifica usato per rappresentare i numeri interi (positivi e negativi)

- se il numero è positivo si converte con il metodo classico
- se il numero è negativo
 - si effettua il CA1
 - si somma +1 al risultato ottenuto con CA1
- Dati n bit si possono rappresentare i numeri dell'intervallo: $[-(2^{n-1}), +(2^{n-1}-1)]_{10}$
- CA2 ha un'unica rappresentazione dello zero (proprio perché l'intervallo è asimmetrico)
- In CA2 i valori negativi hanno MSB = 1

bit	val. assoluto	CA2
0111 1111	127	127
0111 1110	126	126
0000 0010	2	2
0000 0001	1	1
0000 0000	0	0
1111 1111	255	-1
1111 1110	254	-2
1000 0010	130	-126
1000 0001	129	-127
1000 0000	128	-128

Esistono 3 metodi per calcolare il complemento a 2 di un numero X

1. si applica la definizione di complemento alla base $\longrightarrow CA2(x) = 2^n - x$
2. si calcola CA1 e si somma 1 $\longrightarrow CA1(x) = (2^n - 1) - x \longrightarrow CA2(x) = CA1(x) + 1$
3. partendo da destra:
 - trascrivo tutti gli 0 fino al primo 1
 - trascrivo l'1
 - complemento tutti i bit restanti

N.B. il CA2 di un numero negativo dà il corrispondente positivo $\longrightarrow CA2(CA2(x)) = x$

ESEMPIO: rappresentare su 4 bit il numero $-7_{10} \longrightarrow 7_{10} = 111_2$

1. applico la definizione: $2^4 - 7 = 10000 - 111 = 1001_{CA2}$
2. passando da CA1: $111_2 \xrightarrow{4\text{bit}} 0111_{CA1} \xrightarrow{CA1} 1000_{CA1} \xrightarrow{+1} 1001_{CA2}$
 $\begin{array}{c} 111_2 \\ \hline +7 \end{array} \quad \begin{array}{c} 0111_{CA1} \\ \hline +7 \end{array} \quad \begin{array}{c} 1000_{CA1} \\ \hline -7 \end{array} \quad \begin{array}{c} 1001_{CA2} \\ \hline -7 \end{array}$
3. regola pratica: $111_2 \xrightarrow{4\text{bit}} 0111_{CA2} \xrightarrow{CA2} 1001_{CA2}$
 $\begin{array}{c} 111_2 \\ \hline +7 \end{array} \quad \begin{array}{c} 0111_{CA2} \\ \hline +7 \end{array} \quad \begin{array}{c} 1001_{CA2} \\ \hline -7 \end{array}$

Rappresentazione e operazione

- rappresentazione = come sono organizzati i bit
- operazione = procedura di trasformazione dei bit
 - per rappresentare un numero positivo non serve applicare l'operazione di CA2
 - per un numero negativo è necessario applicare l'operazione alla rappresentazione del corrispondente valore positivo

Conversione da CA2 a decimale

- se il numero è positivo (MSB = 0) si converte in base decimale usando il sistema binario puro
- se il numero è negativo (MSB = 1) si applica l'operazione di CA2 a questo valore ottenendo la rappresentazione del corrispondente positivo che si converte in binario puro e si aggiunge il segno "-"

Operazioni Aritmetiche - Modulo e Segno

La somma in MS si esegue confrontando i bit di segno dei due numeri

a) se i bit di segno sono uguali:

- il bit di segno risultante sarà il bit di segno dei due addendi
- eseguo la somma bit a bit (a meno di overflow)

b) se i bit di segno sono diversi:

- confronto i valori assoluti dei due addendi
- il bit di segno risultante sarà il bit di segno dell'addendo con valore assoluto maggiore
- eseguo la differenza tra i bit

La sottrazione in MS si esegue confrontando i bit di segno dei due numeri

a) se i bit di segno sono uguali:

- il bit di segno risultante sarà il bit di segno dell'operando a modulo maggiore
- il risultato avrà modulo pari al modulo della differenza dei moduli degli operandi

b) se i bit di segno sono diversi:

- il bit di segno risultante sarà uguale al bit di segno del minuendo
- il risultato avrà modulo pari alla somma dei due operandi

Osservazioni

- $A - B = A + (-B)$
- Si può avere overflow solo quando:
 - si sommano due operandi con segno concorde
 - si sottraggono due operandi con segno discorde
- L'overflow avviene quando c'è un riporto dalla cifra più significativa del modulo, cioè non si è nella condizione di rappresentare il risultato ottenuto
- Nelle operazioni tra valori rappresentati in MS gli operandi DEVONO essere rappresentati con lo stesso numero di cifre (si aggiungono gli zeri necessari a sinistra del modulo prima del bit di segno)

Operazioni Aritmetiche - Complemento a 2

La somma in complemento a 2

1. Si esegue la somma su tutti i bit degli addendi, segno compreso
2. Un eventuale riporto (carry) oltre il bit di segno (MSB) viene scartato
3. Nel caso gli operandi siano di segno concorde occorre verificare la presenza o meno dell'overflow
 - l'overflow non si presenta mai quando si sommano operandi di segno opposto
 - l'overflow si presenta se: $(+A) + (+B) = -C$ oppure $(-A) + (-B) = +C$
 - un altro modo per vedere se c'è overflow è guardare i riporti nelle ultime due posizioni più significative: se sono diversi c'è overflow

Esempio: $3 + (-8)$

$$\begin{array}{r} (+3) \quad 0000 \ 0011 \\ + (-8) \quad 1111 \ 1000 \\ \hline (-5) \quad 1111 \ 1011 \end{array}$$

$-2 + (-5)$

$$\begin{array}{r} (-2) \quad 1111 \ 1110 \\ + (-8) \quad 1111 \ 1011 \\ \hline (-7) \ 1 \ 1111 \ 1001 \end{array} \quad \text{scarto il carry}$$

La sottrazione in CA2 viene trasformata in somma applicando la regola

$$A - B = A + (-B) \quad \text{ovvero} \quad A - B = A + \text{CA2}(B)$$

Per assicurarsi della correttezza del risultato bisogna verificare l'assenza di overflow

- non si ha overflow se gli operandi hanno segno discorde
- si ha overflow se gli operandi hanno segno concorde e il segno del risultato è discorde con essi

Esempio: $8 - (+5)$

$$\begin{array}{r} (+8) \quad 0000 \ 1000 \\ - (+5) \quad 0000 \ 0101 \\ \hline (+3) \end{array} \quad \rightarrow \text{CA2} \rightarrow \begin{array}{r} 0000 \ 1000 \\ + 1111 \ 1011 \\ \hline 1 \ 0000 \ 0011 \end{array} \quad \text{scarto il carry}$$

Osservazioni

- gli operandi devono essere rappresentati con lo stesso numero di bit
- nell'ipotesi di avere un valore X su n bit e volerne ricavare la rappresentazione su m bit ($m > n$) si attua l'estensione del segno cioè si replica l'MSB negli $(m-n)$ bit più a sinistra

Esempio: si supponga di avere il numero 18 rappresentato in CA2 in 8 bit e volerlo rappresentare in 12

$$+18 = -0001-0010$$

$$-18 = -1110-1110$$

$$+18 = 0000-0001-0010$$

$$-18 = 1111-1110-1110$$

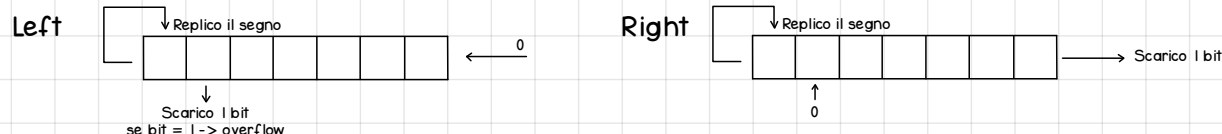
(si aggiungono 0 nella parte più significativa)

(si aggiungono 1 nella parte più significativa)

Esiste un'ulteriore operazione detta shift

- consiste nello spostare (shift) verso destra o verso sinistra la posizione delle cifre di un numero, espresso in una base qualsiasi, inserendo uno zero nelle posizioni lasciate libere
- Left: equivale a moltiplicare il numero per la base
- Right: equivale a dividere il numero per la base

Shift- MS



Shift - CA2



Rappresentazione Eccesso $2^{(n-1)}$

Nella rappresentazione "Eccesso 2^{n-1} " un numero X e rappresentato come segue: $X+2$

- con n bit si rappresenta l'eccesso 2^{n-1}
- l'intervallo e asimmetrico come in CA2 $[-2^{n-1}, +2^{n-1} - 1]$ quindi c'e una semplice rappresentazione dello zero
- REGOLA PRATICA: i numeri in eccesso 2^{n-1} si ottengono da quelli in CA2 complimentando il bit piu significativo

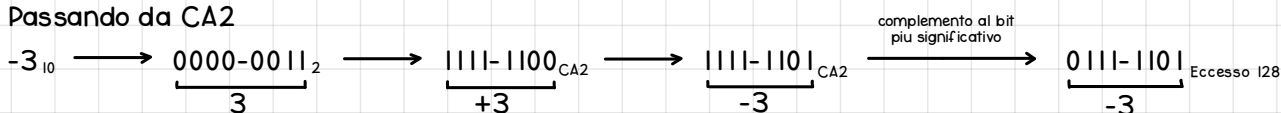
Rappresentazione Eccesso 128

Nella rappresentazione "eccesso 128" (2^7) un numero X e rappresentato come segue: $X+128$

Esempio:

$X=5$	$5 + 128 = 133$	$1000\ 0101_2$
$X=-3$	$-3 + 128 = 125$	$0111\ 1101_2$

Passando da CA2



Osservazioni

- in eccesso 128 i numeri da $[-128, 127]$ si mappano su $[0, 255]$
- in eccesso si usa una convenzione per il bit di segno opposta: 0 per i negativi e 1 per i positivi

Rappresentazione dei numeri reali

I numeri reali possono essere rappresentati come segue:

- virgola fissa (fixed point)
- virgola mobile (floating point)

Virgola fissa

Un sistema di numerazione in virgola fissa è quello in cui:

- Si riserva un numero FISSO di bit per la parte intera e la parte frazionaria
- La posizione della virgola decimale è implicita
- La posizione della virgola decimale è uguale in tutti i numeri

UNSIGNED fixed point: dati N bit a disposizione

- $I < N$ bit per rappresentare la parte intera del numero
- $D = N - I$ per rappresentare la parte decimale del numero

Con questo metodo l'intervallo dei numeri interi rappresentabili è: $[0, 2^I - 1]$

L'intervallo rappresentabile della parte decimale è: $[0, 2^D - 1]$

SIGNED fixed point: dati N bit a disposizione

- 1 bit per il segno del numero da rappresentare
- $I < (N - 1)$ bit per rappresentare la parte intera del numero
- $D = N - (I + 1)$ bit per rappresentare la parte decimale del numero

Con questo metodo l'intervallo dei numeri interi rappresentabili è: $[-2^{I-1}, 2^{I-1} - 1]$

L'intervallo rappresentabile della parte decimale è: $[0, 2^D - 1]$

Supponiamo di volere rappresentare numeri reali con 8 bit. Possiamo decidere di dedicare:

- 1 bit al segno, 3 bit alla parte intera e 4 bit alla parte decimale.
- Oppure 1 bit al segno, 5 bit alla parte intera e 2 bit alla parte decimale.

La scelta è dettata dalle necessità pratiche.

- Se volessimo rappresentare numeri che hanno una parte intera grande (e quindi che hanno un valore I grande) ma una precisione piccola (dedicando meno bit alla parte decimale del numero) sceglieremmo la seconda opzione.
- Se volessimo rappresentare numeri che hanno parte intera piccola ma una grande precisione decimale, sceglieremmo la prima opzione.

Consideriamo il numero $X.YYYY$ in base 2. La conversione di un numero frazionario (base 2) in decimale (base 10) in virgola fissa avviene come ricorrendo alla conversione vista per il sistema binario puro.

- Esempio: consideriamo il numero 101.01_2 , e rappresentiamolo in base 10

$$101.01_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} = 4 + 0 + 1 + 0 + 0.25 = 5.25_{10}$$

Consideriamo il numero $X.YYYY$ in base 10. La conversione di un numero frazionario (base 10) in binario (base 2) in virgola fissa avviene come segue:

1. Si considera la parte intera X e la si riporta in base 2
2. La parte decimale YYYY viene scomposta per moltiplicazioni successive, prendendo la parte intera di ciascun risultato

Convertiamo 9,6234 da base 10 a base 2

parte intera 9 -> 1001

parte dopo la virgola

$$0,6234 * 2 = 1,2468$$

$$0,2468 * 2 = 0,4936$$

$$0,4936 * 2 = 0,9872$$

$$0,9872 * 2 = 1,9744$$

$$0,9744 * 2 = 1,9488$$

$$0,9488 * 2 = 1,8976$$

$$0,8976 * 2 = 1,7952$$

$$0,7952 * 2 = 1,5904$$

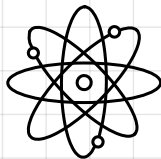
...

Non bastano 8 cifre decimali per rappresentare il numero
C'è quindi un errore di rappresentazione

Svantaggi della rappresentazione in virgola fissa:

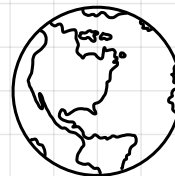
- Rigidità della posizione assegnata alla virgola
Sono quindi fissi i bit per parte intera e parte decimale
- Impatto sulla precisione nel codificare i numeri
Maggiore è il numero di bit riservati per la parte intera, più bassa sarà la precisione nel codificare numeri piccoli
- Spreco di bit per memorizzare zeri
Sia in numeri grandi che in numeri piccoli se ci sono molti zeri si occupano bit per rappresentarli

Virgola mobile



La massa dell'elettrone è

0.00000000000000000000000000000091 kg



La massa della Terra è

59736000000000000000000000 kg

Come possiamo gestire numeri di questo tipo e stabilire univocamente la convenzione sulla posizione della virgola?

Di quale capacità di rappresentazione avremmo bisogno per rappresentare queste quantità in binario

Potremmo semplicemente scrivere il numero indicandone le cifre essenziali

- $9.1 \cdot 10^{-31}$
- $5.9736 \cdot 10^{24}$

Questa notazione all'interno degli elaboratori si definisce virgola mobile (floating point)

In forma generale un numero N si esprime come:

$$N = \pm M \cdot B^{\pm E}$$

M = mantissa
B = base
E = esponente

Più bits per mantissa = maggiore accuratezza

Più bits per esponente = maggiore intervallo

La rappresentazione in virgola mobile usa:

- Un bit per rappresentare il segno s
- Altri bit per rappresentare la mantissa m (chiamata anche significante)
- Altri bit per codificare l'esponente e

s	e	...	e	m	...	m
Segno	Esponente			Mantissa		

Ci sono due forme di rappresentazione:

1. Non normalizzata (arbitraria) $363,4 \cdot 10^{34}$ $36,34 \cdot 10^{35}$
2. Normalizzata cioè 1 cifra (diversa da 0) per la mantissa $3,634 \cdot 10^{36}$ $1,xxx \cdot 2^{yy}$

La notazione scientifica per la base 2:

$$1,xx...xx_2 \cdot 2^{yy...yy_2}$$

dove x rappresenta la parte frazionaria mentre y l'esponente a cui elevare la base 2

- Semplice/singola precisione su 32 bit

1	8	23
Segno	Esponente	Mantissa

- Doppia precisione su 64 bit

1	11	52
Segno	Esponente	Mantissa

Supponiamo che il numero utilizzi il formato a 32 bit

Il bit iniziale 1 non viene memorizzato (poiché è sempre 1 per un numero normalizzato) e viene definito "bit nascosto".

-53.5 viene rappresentato «codificato»

$$-53.5 = (-110101.1) = (-1.101011) \times 2^5$$

1	00000101	101011000000000000000000
segno	esponente	mantissa

Il numero positivo normalizzato più piccolo che rientra in 32 bit è:

$$(1.000000000000000000000000)_2 \cdot 2^{-126} = 2^{-126} \approx 1.18 \cdot 10^{-38}$$

0	10000010	000000000000000000000000
segno	esponente	mantissa

Il numero positivo normalizzato più grande che rientra in 32 bit è:

$$(1.111111111111111111111111)_2 \cdot 2^{127} = (2^{24} - 1) \cdot 2^{104} \approx 3.40 \cdot 10^{38}$$

0	01111111	111111111111111111111111
segno	esponente	mantissa

Standard IEEE 754

Si è reso necessario definire uno standard per la rappresentazione dei numeri in virgola mobile per definire la semantica delle istruzioni in virgola mobile

- IEEE Computer Society (Institute of Electrical and Electronics Engineers) definisce lo "IEEE standard for binary floating arithmetic" noto anche come IEEE 754 nel 1985.
- Specifica il formato, le operazioni, le conversioni tra i diversi formati floating point e quelle tra i diversi sistemi di numerazione, il trattamento delle eccezioni
- Nel 1989 IEEE 754 diventa uno standard internazionale

Lo standard IEEE 754 impone un formato non proprietario, ossia non dipendente dall'architettura del calcolatore

Esponente (8 bit)

- Rappresentato in eccesso 127 (polarizzazione o bias)
- L'intervallo di rappresentazione è $[-127, 128]$
- Se gli 8 bit dell'esponente contengono 10100011 = 163 $\rightarrow$ L'esponente vale $163 - 127 = 36$
- Se gli 8 bit dell'esponente contengono 00100111 = 39 $\rightarrow$ L'esponente vale $39 - 127 = -88$

La mantissa è sempre nella forma:

- 1,XXXXXXXXXX...X

Un numero X in virgola mobile secondo IEEE 754 viene rappresentato come

- $X = (-1)^S \cdot (1 + 0.M) \cdot 2^{E-127}$

Errore assoluto e errore relativo

Rappresentando un numero reale n in virgola mobile si commette un errore di approssimazione.

- In realtà viene rappresentato un numero razionale n' con un numero limitato di cifre significative:

$$\text{ERRORE ASSOLUTO: } e_A = n - n'$$

$$\text{ERRORE RELATIVO: } e_R = e_A / n = (n - n') / n$$

- L'ordine di grandezza dell'errore assoluto dipende dal numero di cifre significative e dall'ordine di grandezza del numero
- L'ordine di grandezza dell'errore relativo dipende solo dal numero di cifre significative

Rappresentazione dei caratteri

Possiamo associare a ogni carattere (quale lettera minuscola, lettera maiuscola, vocale accentata e segno di interpunzione) un numero.

I caratteri possono essere rappresentati in:

- ASCII standard: 1 carattere è rappresentato con 7 bit per un totale di 128 simboli rappresentabili (quali cifre, lettere maiuscole e lettere minuscole);
- ASCII estesa: 1 carattere è rappresentato con 8 bit rappresentabili fino a 256 simboli (i caratteri in più sono usati per esempio per caratteri accentati);
- UNICODE: 1 carattere è rappresentato con un numero maggiore di bit (tra 8 e 32 bit per carattere).

ASCII standard

ASCII standard contiene:

- 26 + 26 lettere (maiuscole + minuscole)
- 10 cifre decimali (da 0 a 9)
- segni di interpunzione
- caratteri di controllo

Le cifre sono ordinate per valore, le lettere maiuscole sono ordinate alfabeticamente, quelle minuscole anche e (e sono a distanza fissa dalle maiuscole)

bits				Column				Row			
b ₄	b ₃	b ₂	b ₁								
0	0	0	0	0	NUL	DLE	SP	0	@	P	~
0	0	0	0	1	SOH	DC1	!	1	A	Q	a
0	0	0	1	0	2	STX	DC2	"	2	B	R
0	0	0	1	1	3	ETX	DC3	#	3	C	S
0	0	1	0	0	4	EOT	DC4	\$	4	D	T
0	0	1	0	1	5	ENQ	NAK	%	5	E	U
0	0	1	1	0	6	ACK	SYN	&	6	F	V
0	0	1	1	1	7	BEL	ETB	'	7	G	W
0	1	0	0	0	8	BS	CAN	(	8	H	X
0	1	0	0	1	9	HT	EM	)	9	I	Y
0	1	0	1	0	10	LF	SUB	*	:	J	Z
0	1	0	1	1	11	VT	ESC	+	;	K	[
0	1	1	0	0	12	FF	FS	,	<	L	\
0	1	1	0	1	13	CR	RS	=	=	M	]
0	1	1	1	0	14	SO	GS	.	>	N	^
0	1	1	1	1	15	SI	US	/	?	O	_

ASCII esteso

Con 1 byte ($2^8=256$) è possibile realizzare 256 diverse combinazioni

La tabella ASCII estesa varia in base alla zona geografica di utilizzo e al software utilizzato

ASCII è un codice accettato da tutti i computer

- Usato dai tempi delle telescriventi durante la prima guerra mondiale
- Tuttavia non considera i caratteri internazionali di numerose lingue straniere
- Per ovviare a tale problematica, è stata introdotta un'ulteriore codifica, ossia UNICODE

ASCII control characters			ASCII printable characters																Extended ASCII characters															
00	NUL	(Null character)	32	space	64	@	96	`	128	Ç	160	à	192	À	224	Ó	256	à																
01	SOH	(Start of Header)	33	!	65	A	97	a	129	ü	161	í	193	Á	225	Ô	257	á																
02	STX	(Start of Text)	34	"	66	B	98	b	130	é	162	ó	194	Â	226	Õ	258	â																
03	ETX	(End of Text)	35	#	67	C	99	c	131	â	163	û	195	Ã	227	Ö	259	ã																
04	EOT	(End of Trans.)	36	\$	68	D	100	d	132	ä	164	ü	196	Ä	228	Ø	260	ä																
05	ENQ	(Enquiry)	37	%	69	E	101	e	133	å	165	ñ	197	Å	229	Ù	261	å																
06	ACK	(Acknowledgement)	38	&	70	F	102	f	134	ä	166	ª	198	Ä	230	Ú	262	æ																
07	BEL	(Bell)	39	'	71	G	103	g	135	ç	167	»	199	Å	231	Û	263	ç																
08	BS	(Backspace)	40	(	72	H	104	h	136	ê	168	¿	200	Ê	232	Ü	264	è																
09	HT	(Horizontal Tab)	41	)	73	I	105	i	137	ë	169	Ø	201	Ë	233	Ý	265	é																
10	LF	(Line feed)	42	*	74	J	106	j	138	è	170	•	202	È	234	ÿ	266	ê																
11	VT	(Vertical Tab)	43	+	75	K	107	k	139	í	171	¼	203	É	235	ÿ	267	ë																
12	FF	(Form feed)	44	,	76	L	108	l	140	î	172	½	204	Ê	236	ÿ	268	ì																
13	CR	(Carriage return)	45	-	77	M	109	m	141	ï	173	¾	205	Ë	237	ÿ	269	í																
14	SO	(Shift Out)	46	.	78	N	110	n	142	À	174	•	206	•	238	ÿ	270	î																
15	SI	(Shift In)	47	/	79	O	111	o	143	Á	175	•	207	•	239	ÿ	271	ï																
16	DLE	(Data link escape)	48	0	80	P	112	p	144	Â	176	•	208	•	240	ÿ	272	ò																
17	DC1	(Device control 1)	49	1	81	Q	113	q	145	Ã	177	•	209	•	241	ÿ	273	ó																
18	DC2	(Device control 2)	50	2	82	R	114	r	146	Ä	178	•	210	•	242	ÿ	274	ô																
19	DC3	(Device control 3)	51	3	83	S	115	s	147	Å	179	•	211	•	243	ÿ	275	õ																
20	DC4	(Device control 4)	52	4	84	T	116	t	148	Æ	180	•	212	•	244	ÿ	276	ö																
21	NAK	(Negative acknowl.)	53	5	85	U	117	u	149	•	181	•	213	•	245	ÿ	277	÷																
22	SYN	(Synchronous idle)	54	6	86	V	118	v	150	•	182	•	214	•	246	ÿ	278	ø																
23	ETB	(End of trans. block)	55	7	87	W	119	w	151	•	183	•	215	•	247	ÿ	279	ù																
24	CAN	(Cancel)	56	8	88	X	120	x	152	•	184	•	216	•	248	ÿ	280	ú																
25	EM	(End of medium)	57	9	89	Y	121	y	153	•	185	•	217	•	249	ÿ	281	û																
26	SUB	(Substitute)	58	:	90	Z	122	z	154	•	186	•	218	•	250	ÿ	282	ü																
27	ESC	(Escape)	59	;	91	[	123	{	155	•	187	•	219	•	251	ÿ	283	ý																
28	FS	(File separator)	60	<	92	\	124	}	156	•	188	•	220	•	252	ÿ	284	ÿ																
29	GS	(Group separator)	61	=	93	]	125	}	157	•	189	•	221	•	253	ÿ	285	ÿ																
30	RS	(Record separator)	62	>	94	^	126	~	158	•	190	•	222	•	254	ÿ	286	ÿ																
31	US	(Unit separator)	63	?	95	_			159	•	191	•	223	•	255	ÿ	287	ÿ																
	DEL	(Delete)																nbsp																

UNICODE

È un'evoluzione dello standard ASCII che codifica tutti i caratteri utilizzati nelle principali lingue del mondo

- Indipendente dalla lingua, dal sistema operativo e dal programma utilizzato
- Inizialmente rappresentato come una codifica su 16 bit, ma poi esteso a 24 e 32 bit

Un carattere UNICODE è caratterizzato dal suo codice numerico, detto code point, solitamente rappresentato con 8 cifre esadecimali

- Con UNICODE, è possibile creare e gestire senza troppa pena documenti multilingue
- In particolare, tutti gli standard W3C (incluso HTML) supportano UNICODE
- A marzo 2024 è stato presentato l'ultima versione UNICODE 16.0

UNICODE può codificare 4.294.967.296 caratteri distinti, ogni carattere occupa 32 bit (contro gli 8 delle altre codifiche); i documenti richiedono quindi 4 volte lo spazio

- La quasi totalità dei documenti usa da 60 a 1000 caratteri, per cui basterebbero da 6 a 10 bit.
- Per ovviare a questo problema, e garantire maggiore compatibilità con S.O. e applicazioni che non sono in grado di gestire 32 bit per carattere, UNICODE definisce vari formati di codifica più compatti

UTF-8 (8-bit UCS/Unicode Transformation Format)

È una codifica a lunghezza variabile fra 1 sequenza di valori a 8 bit e 1 sequenza di caratteri UNICODE

- I primi 128 caratteri di UNICODE (0-7F), equivalenti ai caratteri ASCII, sono codificati con il loro codice "naturale"
- Tutti gli altri caratteri sono codificati con due, tre o quattro valori a 8 bit (byte)

CIRCUITI DIGITALI E LOGICA COMBINATORIA

Introduzione

- I circuiti logici sono realizzati come circuiti integrati realizzati su chip di silicio (piastrina)
- Porte (gate) e fili depositati su chip di silicio, inseriti in un package e collegati all'esterno con un certo insieme di pin (piedini)
- I circuiti integrati si distinguono per grado di integrazione (da singole porte indipendenti a circuiti più complessi)

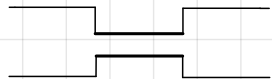
Integrazione

- SSI (Small Scale Integrated): 1-10 porte
Con tecnologia SSI, i circuiti integrati contenevano poche porte, direttamente collegate ai pin esterni
- MSI (Medium Scale Integrated): 10-100 porte
Con tecnologia MSI, i circuiti integrati contenevano alcuni componenti base (circuiti comunemente usati nel progetto di un computer)
- LSI (Large Scale Integrated): 100-100.000 porte
- VLSI (Very Large Scale Integrated): > 100.000 porte
Con tecnologia VLSI, i circuiti integrati possono oggi contenere una CPU completa o più (microprocessore)

Nell'elettronica digitale sia gli ingressi che le uscite possono assumere solo i valori di segnale alto (1 per convenzione) o basso (0 per convenzione).

In un circuito digitale i valori binari sono ottenuti tramite discretizzazione dei segnali:

- Falso: segnali con voltaggio basso ≤ 1 0/falso/[0...1] Volt
- Vero: segnali con voltaggio più alto > 1 1/vero/[2...5] Volt



- Un circuito (o rete) combinatoria è quel circuito il cui lo stato delle uscite dipende solo dalla funzione logica applicata allo stato istantaneo (cioè in un determinato istante di tempo) delle sue entrate.
- Un circuito (o rete) sequenziale è quel circuito il cui lo stato delle uscite non dipende solo dalla funzione logica applicata ai suoi ingressi, ma anche sulla base di valori pregressi collocati in memoria.

Porte logiche

Le porte logiche sono i componenti elettronici che permettono di svolgere le operazioni logiche primitive oltre che a quelle direttamente derivate.

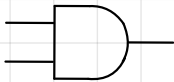
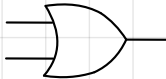
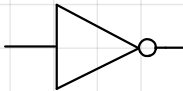
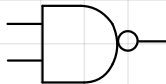
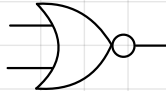
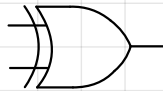
- Le porte logiche che realizzano le operazioni principali dell'algebra booleana
- Una porta logica è un circuito elettronico che dati dei segnali 0 e 1 in input produce un segnale in output ottenuto effettuando una operazione booleana sugli ingressi

Le porte logiche hanno n input e generalmente 1 output

- A ogni combinazione di valori in ingresso corrisponde una e solo una combinazione in di valori uscita
- Dati gli input, l'output corrispondente appare quasi istantaneamente

Porte logiche fondamentali: AND, OR, NOT

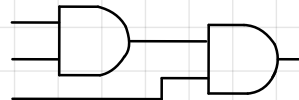
Porte logiche derivate: NAND, NOR, XOR

La porta logica AND svolge l'operazione logica di AND tra due bit, detta anche prodotto logico. • Considerando due entrate A e B, l'uscita $A \cdot B$ è data da:		<table><tr><th>A</th><th>B</th><th>$A \cdot B$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$A \cdot B$	0	0	0	0	1	0	1	0	0	1	1	1					
A	B	$A \cdot B$																				
0	0	0																				
0	1	0																				
1	0	0																				
1	1	1																				
La porta logica OR svolge l'operazione logica di OR tra due bit, detta anche somma logica. • Considerando due entrate A e B, l'uscita $A + B$ è data da:		<table><tr><th>A</th><th>B</th><th>$A + B$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	$A + B$	0	0	0	0	1	1	1	0	1	1	1	1					
A	B	$A + B$																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	1																				
La porta logica NOT svolge l'operazione logica di NOT su un bit, detta anche negazione logica. • Considerando un'entrata A, l'uscita $\bar{A}$ è data da:		<table><tr><th>A</th><th>not A</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	not A	0	1	1	0														
A	not A																					
0	1																					
1	0																					
La porta logica NAND svolge l'operazione logica di NOT sul bit risultante dall'operazione AND sui bit in ingresso.		<table><tr><th>A</th><th>B</th><th>$A \cdot B$</th><th>A nand B</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$A \cdot B$	A nand B	0	0	0	1	0	1	0	1	1	0	0	1	1	1	1	0
A	B	$A \cdot B$	A nand B																			
0	0	0	1																			
0	1	0	1																			
1	0	0	1																			
1	1	1	0																			
La porta logica NOR svolge l'operazione logica di NOT sul bit risultante dall'operazione OR sui bit in ingresso.		<table><tr><th>A</th><th>B</th><th>$A + B$</th><th>A nor B</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$A + B$	A nor B	0	0	0	1	0	1	1	0	1	0	1	0	1	1	1	0
A	B	$A + B$	A nor B																			
0	0	0	1																			
0	1	1	0																			
1	0	1	0																			
1	1	1	0																			
La porta logica XOR opera come disgiunzione esclusiva tra due input.		<table><tr><th>A</th><th>B</th><th>$A \text{ xor } B$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	$A \text{ xor } B$	0	0	0	0	1	1	1	0	1	1	1	0					
A	B	$A \text{ xor } B$																				
0	0	0																				
0	1	1																				
1	0	1																				
1	1	0																				

Porte con piu ingressi

Ad eccezione della porta NOT, le altre porte logiche possono esistere anche ad N ingressi (2, 3, 4, ..., N).

- Queste porte svolgono l'operazione logica associata su N bit invece che su 2.
- Sono particolarmente comode nella rappresentazione grafica dei circuiti logici.
- Nella pratica, cioe nella realizzazione di circuiti, se si hanno a disposizione solo porte a 2 ingressi, e possibile realizzare porte a N ingressi collegando a cascata tra loro porte a 2 ingressi.
- Un AND a 3 ingressi si può creare usando 2 AND a 2 ingressi come segue:



Osservazioni

Qualunque funzione logica puo essere costruita usando le porte logiche AND, OR e NOT.

Le porte NOR e NAND che svolgono la funzione di inverter, sono definite universali.

- NAND OR
 $A \text{ nand } B = \text{not}(A \text{ and } B) = (\text{not } A) \text{ or } (\text{not } B)$
- NAND NOT
 $A \text{ nand } 1 = \text{not}(A \text{ and } 1) = (\text{not } A)$

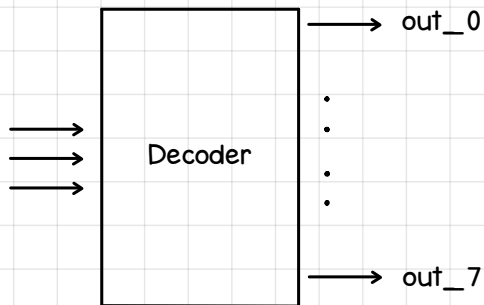
Decoder

Un decoder è un componente elettronico caratterizzato dall'avere n ingressi e 2^n uscite.

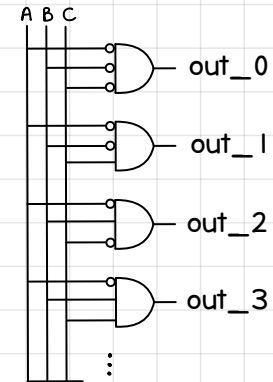
- Lo scopo del decoder è di impostare allo stato alto l'uscita corrispondente alla conversione in base 10 della codifica binaria a n bit ricevuta in input (e di impostare allo stato basso tutte le altre).

In pratica:

- gli n input sono interpretati come un numero unsigned
- se questo numero rappresenta il numero i , allora
- solo il bit in output di indice i ($i=0, 1, \dots, 2^n-1$) verrà posto ad 1
- Tutti gli altri verranno posti a 0



A	B	C	0	1	2	3	4	5	6	7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

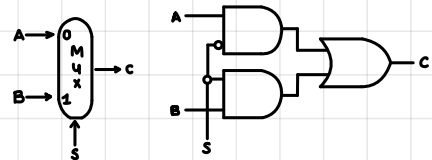


Multiplexer

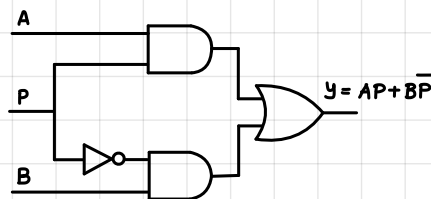
Un multiplexer, detto anche selettore, è un componente elettronico caratterizzato da:

- $2n$ entrate principali
- n entrate di controllo (selettore)
- 1 uscita

Esempio: $C = (A \cdot \bar{S}) + (B \cdot S)$

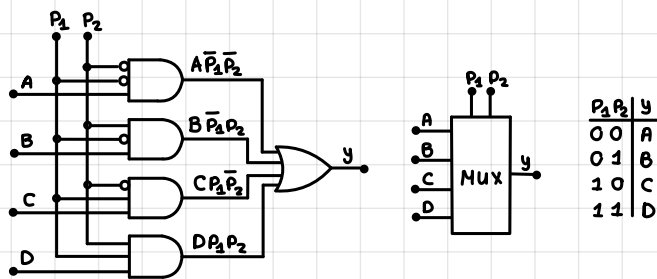


Il valore del selettore determina quale input diviene output



P	Y
0	B
1	A

A=0, B=0, P=1	A=1, B=0, P=1	A=1, B=1, P=1	A=0, B=1, P=1
A=0 AND P=1 >> 0 B=0 AND notP=0 >> 0 0 OR 0 >> 0	A=1 AND P=1 >> 1 B=0 AND notP=0 >> 0 1 OR 0 >> 1	A=1 AND P=1 >> 1 B=1 AND notP=0 >> 0 0 OR 1 >> 1	A=0 AND P=1 >> 0 B=1 AND notP=0 >> 1 0 OR 1 >> 1



Implementa le funzione logica:
 $Y = A\bar{P}_1\bar{P}_2 + B\bar{P}_1P_2 + CP_1\bar{P}_2 + DP_1P_2$

Se un multiplexer riceve in input n segnali, esso necessita di $\log_2 n$ selettori, e consista di:

1. Un decoder che genererà n segnali
2. Un array di n porte logiche AND
3. Un'unica porta logica OR

$$n = 2^m \iff m = \log_2 n$$

Logiche a 2 livelli e PLA

Utilizzando le porte logiche AND, OR e NOT è possibile implementare qualunque funzione logica più complessa.

- Possiamo creare logiche a due livelli:
- Somma di prodotti: somma logica (OR) di prodotti (AND)
- Prodotto di somme: prodotto (AND) di somme (OR)

Consideriamo la seguente tabella di verità per D, dati gli ingressi A, B, C ed esprimiamola come somma di prodotti

$$\bar{A} \cdot \bar{B} \cdot C \quad \bar{A} \cdot B \cdot \bar{C} \quad A \cdot \bar{B} \cdot \bar{C} \quad A \cdot B \cdot C$$

Possiamo descrivere la funzione logica per D così
 $D = (\bar{A} \cdot \bar{B} \cdot C) + (\bar{A} \cdot B \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot B \cdot C)$

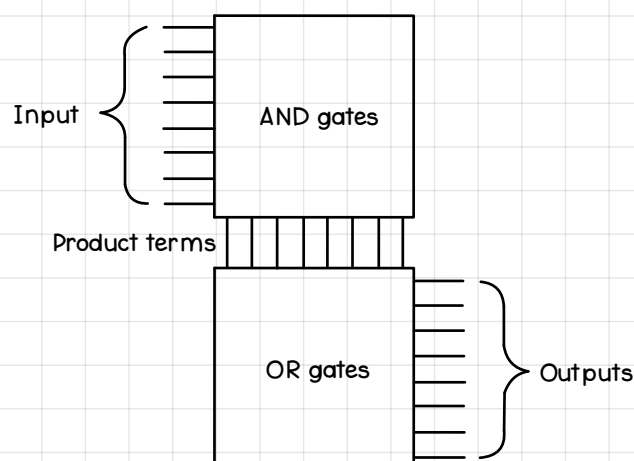
Inputs			Output
A	B	C	D
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

La somma di prodotti corrisponde ad una implementazione comunemente nota come Programmable Logic Array (PLA):

- Un insieme di input
- I corrispondenti input complementati (mediante inverter) per poter gestire più uscite
- Una logica a due stage:

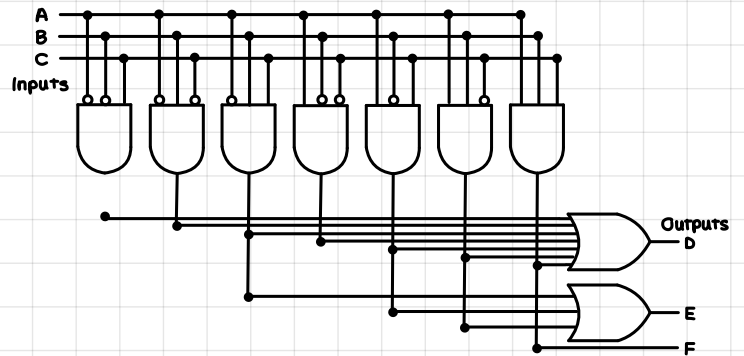
Primo stage: un array di porte logiche AND (prodotto)

Secondo stage: un array di porte logiche OR (somma)



Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

PLA corrispondente delle somme di prodotto



$$D = (\bar{A} \cdot \bar{B} \cdot C) + (\bar{A} \cdot B \cdot \bar{C}) + (\bar{A} \cdot B \cdot C) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot \bar{C}) + (A \cdot B \cdot C)$$

$$E = (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot \bar{C}) + (A \cdot B \cdot C)$$

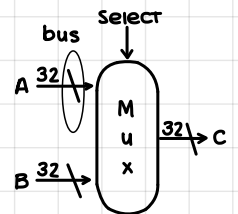
$$F = (A \cdot B \cdot C)$$

$(\bar{A} \cdot \bar{B} \cdot \bar{C})$ non ha output

Array di elementi logici

La maggior parte delle operazioni vengono svolte su 32 bit, mettendo in luce la necessità di creare array di elementi logici.

Un bus è una collezione di linee di input trattate come un singolo segnale. In questo modo un multiplexer con un bus da 32 bit si comporta come un array da 32 multiplexer da 1 bit



ALU (Arithmetic Logic Unit)

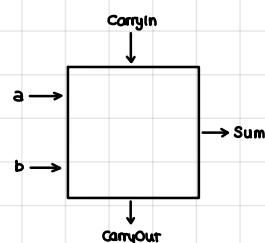
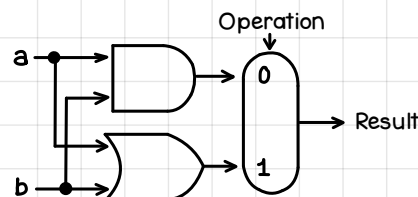
L'Arithmetic Logic Unit è la parte del processore che svolge le operazioni aritmetico-logiche, e un insieme di circuiti combinatori che implementa:

- Operazioni aritmetiche: es somma e sottrazione
- Operazioni logiche: es AND e OR

Blocchi di base per costruire l'ALU:

- AND gate
- OR gate
- Inverter
- Multiplexor

ALU ad 1 bit che implementa AND e OR



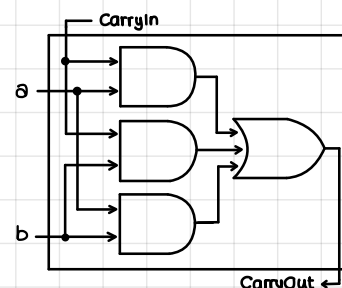
Operazioni aritmetiche

$$\begin{array}{r}
 0000000000000000-0001 \quad 1111 \quad 0011 \quad 0100 \quad \text{carry} \\
 0000000000000000-0000 \quad 1011 \quad 1100 \quad 1011 \quad + \\
 0000000000000000-0000 \quad 0110 \quad 1001 \quad 1010 \quad = \\
 \hline
 0000000000000000-0001-0010-0110-0101
 \end{array}$$

A	B	CarryIn	CarryOut	Somma
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

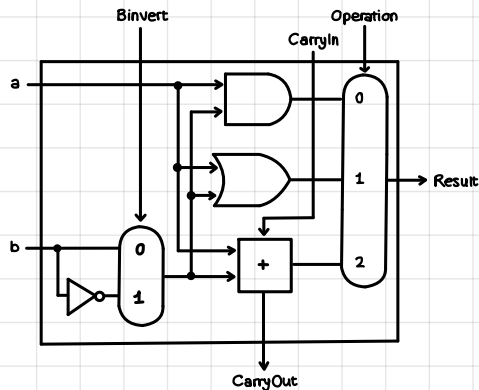
$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b) + (a \cdot b \cdot \text{CarryIn})$$

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$$



Se settiamo il carryIn a 1 possiamo sommare $a+b+1$

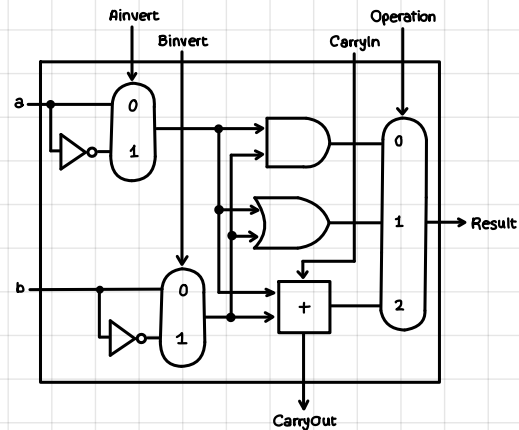
Se neghiamo b possiamo ottenere la sottrazione (in CA2) $a - b = a + (\bar{b} + 1)$



Come si implementa anche NOR? $\text{NOT}(a+b)$

De Morgan:

- $\text{NOT}(a \text{ OR } b) = (\text{NOT } a) \text{ AND } (\text{NOT } b) \rightarrow \overline{a+b} = \bar{a} \cdot \bar{b}$
- $\text{NOT}(a \text{ AND } b) = (\text{NOT } a) \text{ OR } (\text{NOT } b) \rightarrow \overline{a \cdot b} = \bar{a} + \bar{b}$

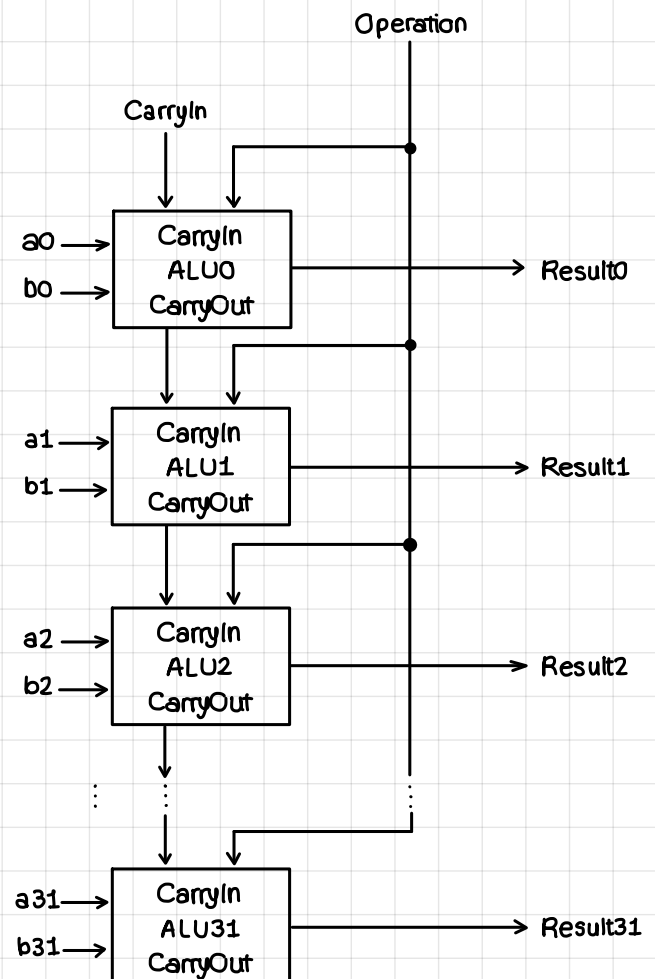


Ripple Carry - ALU a 32 bit

Per combinare più ALU a 1 bit tramite il ripple carry e creare un'ALU a 32 bit, ogni ALU a 1 bit viene utilizzata per eseguire l'operazione su un bit specifico dei due numeri binari che si desidera sommare o sottrarre. Il ripple carry viene quindi utilizzato per propagare eventuali riporti generati da una posizione all'altra, permettendo così di eseguire l'operazione su tutti i 32 bit contemporaneamente.

Ecco un'esemplificazione semplificata del processo:

1. Ogni ALU a 1 bit riceve due bit in input dai due numeri binari che si vogliono sommare o sottrarre, insieme al riporto in ingresso (se presente) dalla posizione meno significativa precedente
2. L'ALU a 1 bit esegue l'operazione specifica (ad esempio, somma o sottrazione) sui due bit in ingresso, generando anche un eventuale riporto in uscita se il risultato supera il valore di un bit
3. Il riporto in uscita dall'ALU a 1 bit viene poi passato all'ALU a 1 bit successiva per la posizione successiva, e così via, fino a raggiungere la posizione più significativa
4. Se il riporto dalla posizione più significativa è generato, questo viene considerato come il riporto finale della somma o sottrazione



In questo modo, utilizzando il ripple carry, le operazioni su tutti i 32 bit possono essere eseguite contemporaneamente attraverso l'intera ALU a 32 bit. Tuttavia, questo metodo può richiedere più cicli di clock per completare l'operazione rispetto a tecniche più avanzate che minimizzano la dipendenza del carry tra i vari bit.

ALU su 32 bit - SLT (operazioni di confronto)

SLT - Set on Less Than

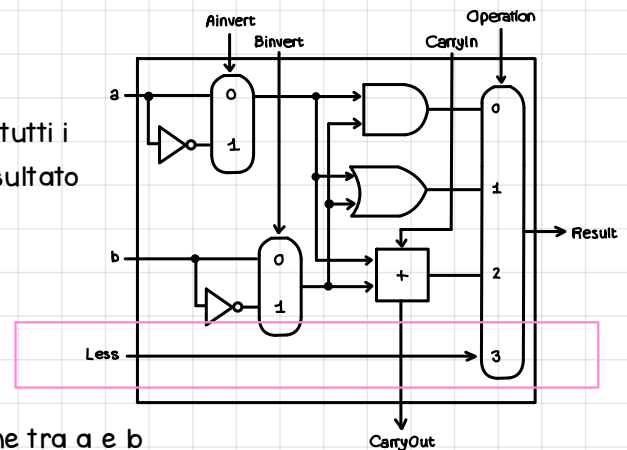
- slt registro 1, registro 2, registro 3
- Poni registro 1 = 1 se il valore contenuto in registro 2 è minore del valore contenuto in registro 3
- Altrimenti se il valore contenuto in registro 2 non è minore del valore contenuto in registro 3, allora poni registro 1 = 0
- Risultato: 1 se $a < b$, 0 altrimenti

Per eseguire questa istruzione si devono poter azzerare tutti i bit dal bit-1 al bit-31 ed assegnare al bit-0 il valore il risultato dell'operazione di slt.

$$a < b \Leftrightarrow a - b < 0$$

$\Leftrightarrow$ la sottrazione $(a - b)$ è negativa

$\Leftrightarrow$ bit 31 della sottrazione $(a - b) = 1$



Per poter realizzare il confronto si effettua la sottrazione tra a e b

- Se $a - b$ è minore di 0, allora $a < b$ (per l'istruzione slt)
-Il risultato sarà 00...01
- Se $a - b$ è maggiore di 0, allora $a > b$ (per l'istruzione slt)
-Il risultato sarà 00...00

ALU1-ALU30:

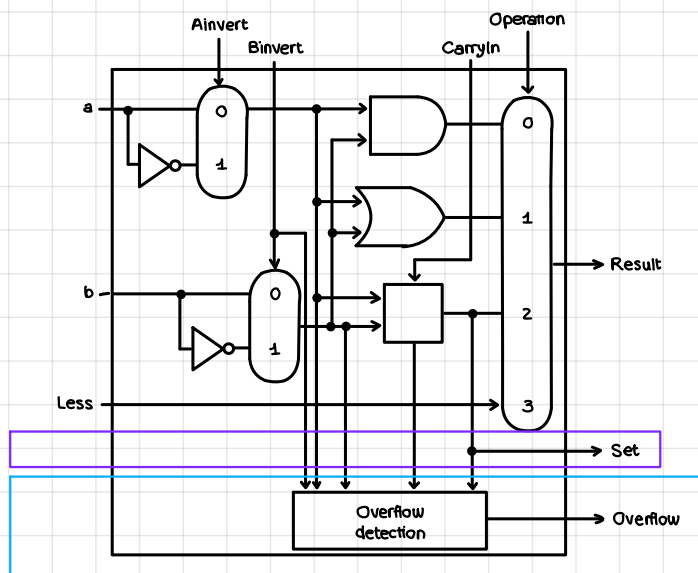
- $\text{CarryIn}_i = \text{CarryOut}_{i-1}$
- $\text{Less} = 0$

ALU0:

- $\text{CarryIn} = 1$ (per sottrazione)
- Less : vedi lucido successivo

ALU31:

- $\text{CarryIn}_{31} = \text{CarryOut}_{30}$
- $\text{Less} = 0$



Casi di overflow: Casi di overflow:

- $(A - B) > 0$ e bit 31 di $(A - B) = 1$ oppure
- $(A - B) < 0$ e bit 31 di $(A - B) = 0$

$$\text{bit 31}(A) = 0$$

$$\text{bit 31}(B) = 1$$

$$\text{bit 31}(A - B) = 1$$

$$\text{bit 31}(A) = 1$$

$$\text{bit 31}(B) = 0$$

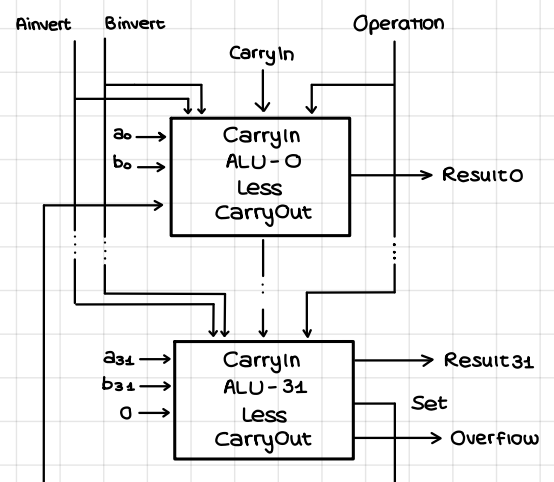
$$\text{bit 31}(A - B) = 0$$

Controllo overflow:

$$\text{overflow} = \bar{a} \cdot b \cdot \text{Result} + a \cdot \bar{b} \cdot \text{Result}$$

ALU su 32 bit - con SLT + overflow

- Per sottrazioni: Binvert e $\text{CarryIn} = 1$
- Per addizioni e operazioni logiche:
 Binvert e $\text{CarryIn} = 0$
- Unificare i due segnali in uno: Bnegate



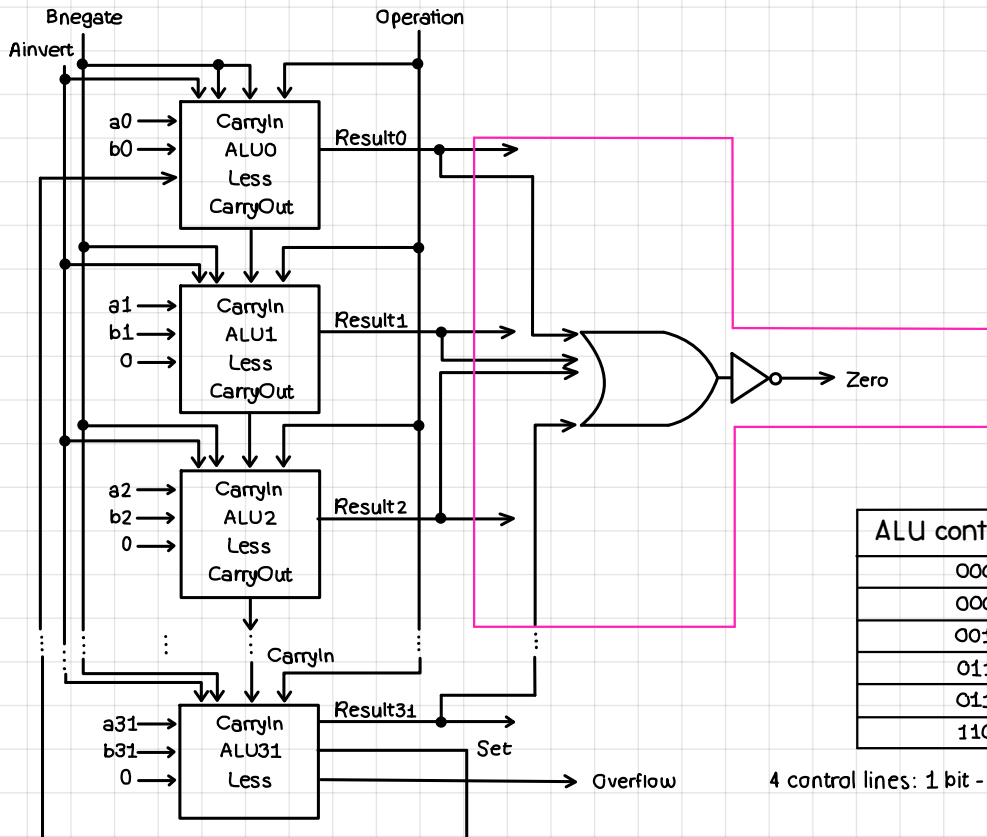
ALU su 32 bit - BEQ (operazioni di confronto)

BEQ: branch-on-equal

- beq registro I, registro2, L I
- Vai all'istruzione etichettata con L I se il valore contenuto in registro I è uguale al valore contenuto in registro2

Per verificare l'uguaglianza di a e b: sottrazione • $a-b=0 \iff a=b$

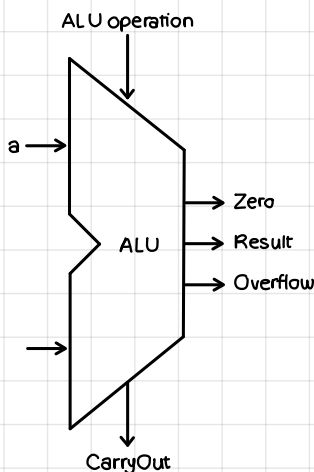
- Se $a=b$ allora dobbiamo mettere a 1 la linea di output usata per l'uguaglianza
- Soluzione: OR di tutte le uscite e poi negare il risultato $\rightarrow Zero = (\text{Result-31} + \text{Result-31} + \dots + \text{Result-0})$



ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

4 control lines: 1 bit - Ainvert 1 bit - Binvert 2 bits - operation

ALU: rappresentazione simbolica



Circuiti combinatori vs sequenziali

- circuiti combinatori= sono in grado di calcolare funzioni che dipendono solo dai dati in input
- circuiti sequenziali= sono in grado di calcolare funzioni che dipendono anche da uno stato
→ ovvero, che dipendono anche da informazioni memorizzate in elementi di memoria interni

Tipologie di circuiti sequenziali

Esistono due famiglie di circuiti digitali sequenziali:

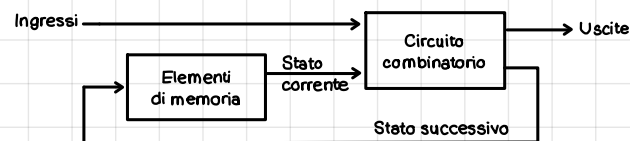
- asincroni= non fanno uso di clock (come l'SR-latch)
non so quando potrebbe avvenire il funzionamento
- sincroni= necessitano di clock (come il Flip-Flop)
funziona tutto in un periodo di clock

Circuiti sequenziali

I circuiti sequenziali sono formati da:

- Elementi di memoria (di vario tipo)= memorizzano informazione
- Reti combinatorie= elaborano informazione

Un circuito sequenziale ha, in ogni dato istante, uno stato determinato dai bit memorizzati

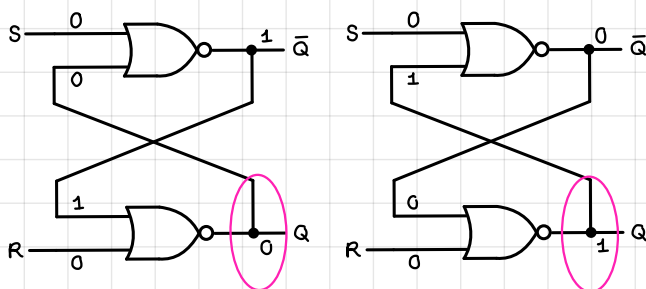
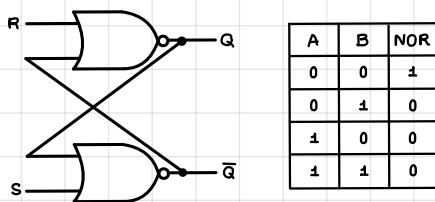


- Per realizzare circuiti sequenziali è necessario un elemento di memoria per memorizzare lo stato
- Possiamo organizzare le porte logiche in modo da realizzare un elemento di memoria? Sì, un elemento in grado di memorizzare un singolo bit e il latch

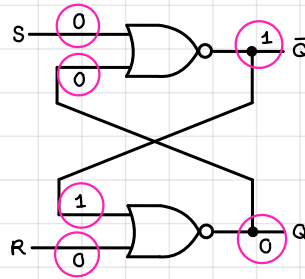
S-R latch

E' un circuito composto da due porte NOR concatenate

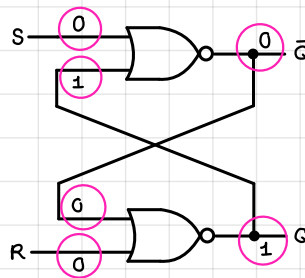
S= set R= reset



Input		Stato Interno Old Q	Output	
S	R		Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
0	1	1/0	0	1
1	0	1/0	1	0
1	1	1/0	0	0

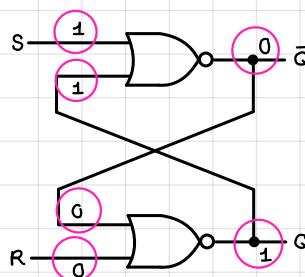


Input		Stato Interno Old Q	Output	
S	R		Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
0	1	1/0	0	1
1	0	1/0	1	0
1	1	1/0	0	0

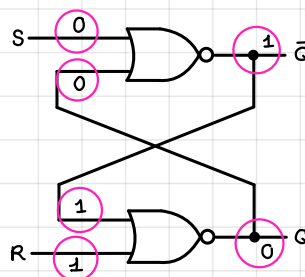


La combinazione $(S, R) = (0, 0)$ viene detta combinazione di riposo, perché semplicemente mantiene il valore memorizzato in precedenza

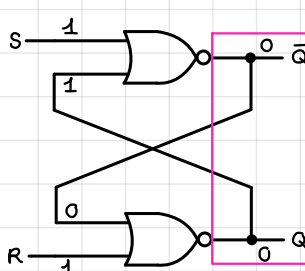
Input		Stato Interno Old Q	Output	
S	R		Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
0	1	1/0	0	1
1	0	1/0	1	0
1	1	1/0	0	0



Input		Stato Interno Old Q	Output	
S	R		Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
0	1	1/0	0	1
1	0	1/0	1	0
1	1	1/0	0	0



Input		Stato Interno Old Q	Output	
S	R		Q	$\bar{Q}$
0	0	0	0	1
0	0	1	1	0
0	1	1/0	0	1
1	0	1/0	1	0
1	1	1/0	0	0



La configurazione $(S, R) = (1, 1)$ viola la proprietà di complementarietà di Q e $\bar{Q}$, può portare ad una configurazione instabile

- (S, R) sono di solito calcolati da un circuito combinatorio
- l'output del circuito diventa stabile dopo un certo intervallo di tempo che dipende dal numero di porte attraversate
- bisogna evitare che durante questo intervallo, gli output intermedi del circuito vengano memorizzati

Latch e clock

Soluzione: clock

- Usiamo un segnale a gradino, il cui periodo viene scelto abbastanza grande da assicurare la stabilità degli output
- Usiamo il clock per abilitare la scrittura nei latch
 - Il clock determina il ritmo dei calcoli e delle relative operazioni di memorizzazione
- Il circuito diventa sincrono
- Il segnale di clock è fondamentale per le reti sequenziali che sono caratterizzate da uno stato
- Il clock è definito come un segnale (onda quadra) con un periodo predeterminato e costante
- Caratterizzato da un periodo T (oppure ciclo) di clock e dalla frequenza F (definita come $1/\text{periodo}$) e misurata in Hertz $F = 1/T$

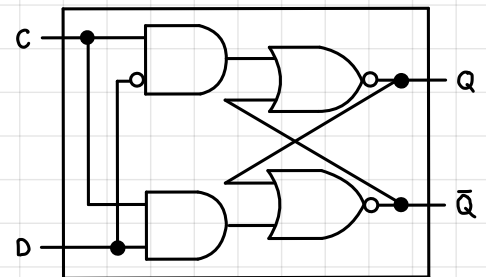


D Latch

- Latch sincronizzato con il clock
- Il clock garantisce che il latch cambi stato solo in certi momenti

$D=1$ corrisponde al setting
 $S=1$ e $R=0$

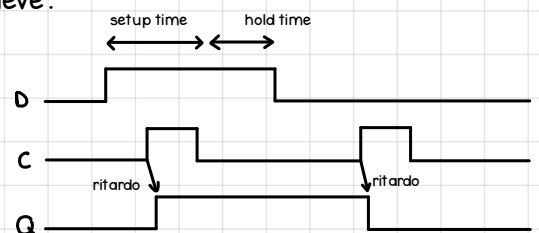
$D=0$ corrisponde al resetting
 $S=0$ e $R=1$



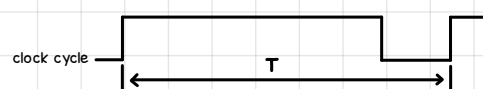
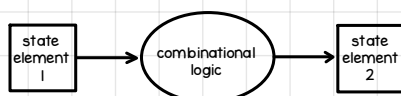
- Quando il clock è deasserted non viene memorizzato nessun valore:
 $S=0$ e $R=0$ (viene mantenuto il valore precedentemente memorizzato)
- Quando il clock è asserted viene memorizzato un valore (in funzione del valore di D)

Il segnale D , ottenuto come output di un circuito combinatorio deve:

- essere già stabile quando C diventa asserted
- rimanere stabile per tutta la durata del livello alto di C (Setup time)
- rimanere stabile per un altro periodo di tempo per evitare malfunzionamenti (Hold time)



Il periodo di clock T deve essere scelto abbastanza lungo affinché l'output del circuito comb. si stabilizzi



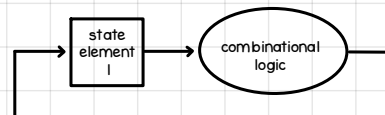
Trasparenza del latch

Il D-latch è caratterizzato dal seguente comportamento:

- durante l'intervallo alto del clock il valore del segnale di ingresso D viene memorizzato nel latch
- il valore di D si propaga quasi immediatamente all'uscita Q
- anche le eventuali variazioni di D si propagano quasi immediatamente, con il risultato che Q può variare più volte durante l'intervallo alto del clock
- solo quando il clock torna a zero Q si stabilizza
- durante l'intervallo basso del clock il latch non memorizza

D latch - input/output

Supponiamo che l'elemento di memoria debba essere usato sia come input che come output durante lo stesso ciclo di clock. Funzionerebbe il D Latch?



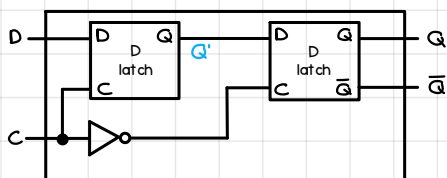
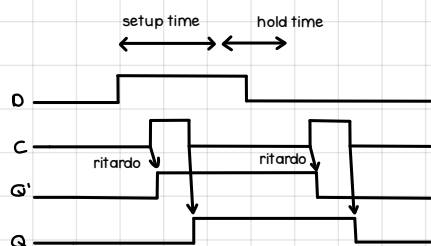
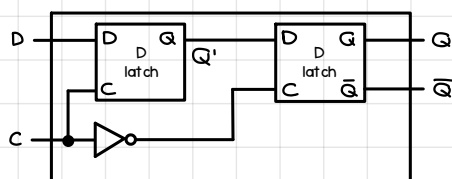
Metodologie di timing

Fino ad ora abbiamo visto una metodologia di timing detta level-triggered che avviene sul livello alto (o basso) del clock. Esiste però una metodologia di timing chiamata edge-triggered che avviene sul fronte di salita (o di discesa) del clock

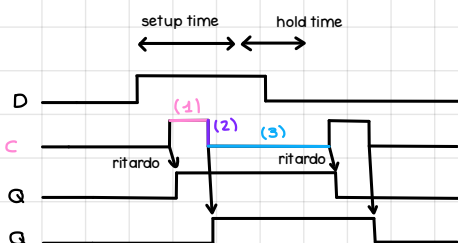
- la memorizzazione avviene istantaneamente
- l'eventuale segnale di ritorno "sporco" non fa in tempo ad arrivare a causa dell'istantaneità della memorizzazione

D Flip-Flop

- Il D Flip-flop è usabile come input e output durante lo stesso ciclo di clock
- Realizzato ponendo in serie 2 D-latch: il primo viene detto master e il secondo slave



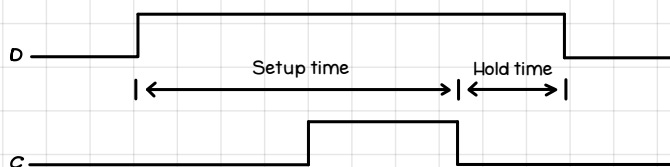
(1) Il primo latch è aperto e pronto per memorizzare D. Il valore memorizzato Q' fluisce fuori, ma il secondo latch è chiuso → nel circuito combinatorio a valle entra ancora il vecchio valore di Q



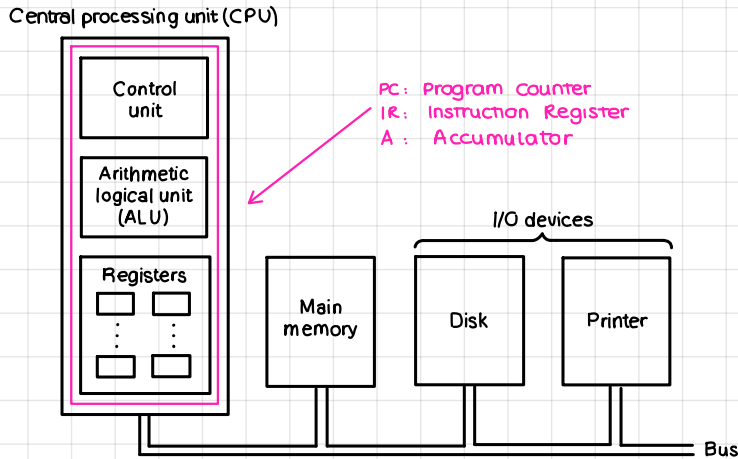
(2) Il segnale del clock scende, e in questo istante il secondo latch viene aperto per memorizzare il valore di Q'

(3) Il secondo latch è aperto, memorizza D (Q'), e fa fluire il nuovo valore Q nel circuito a valle. Il primo latch è invece chiuso, e non memorizza niente

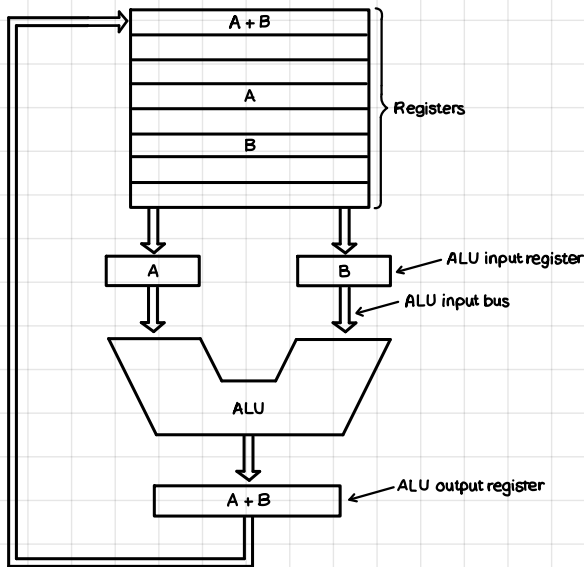
Il segnale D deve essere attivo per un periodo abbastanza lungo: setup time (prima del clock edge) + hold time (dopo il clock edge)



Organizzazione di un calcolatore: richiami



Datapath e Register file

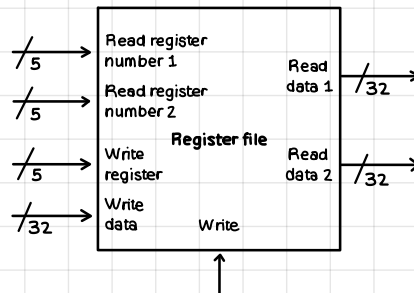


Un registro è costituito da n flip-flop

- Nel MIPS ogni registro è di 1 word = 4 byte = 32 bit

I registri sono organizzati in un Register File

- Il Register File del MIPS ha 32 registri ($32 \times 32 = 1024$ flip-flop)
- Il Register File permette la lettura di 2 registri e la scrittura di 1 registro



Read Reg 1 # (5 bit)

- numero del 1o registro da leggere

Read Reg2 # (5 bit)

- numero del 2o registro da leggere

Read data 1 (32 bit)

- valore del 1o registro, letto sulla base di Read Reg 1 #

Read data 2 (32 bit)

- valore del 2o registro, letto sulla base di Read Reg2 #

Write Reg # (5 bit)

- numero del registro da scrivere

Write data (32 bit)

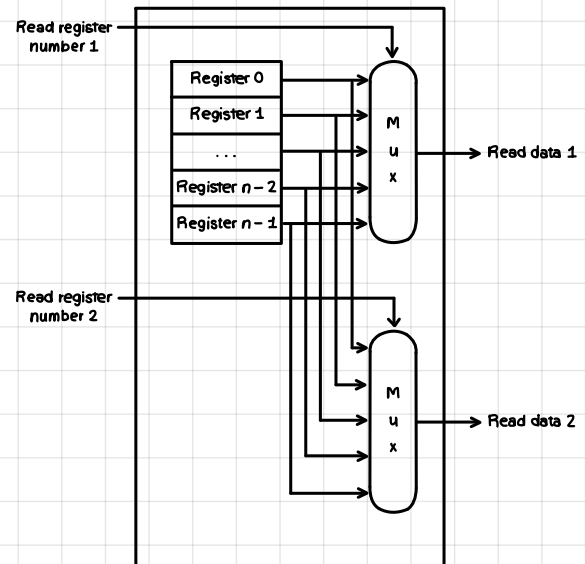
- valore da scrivere nel registro Write Reg #

Write

- segnale di controllo messo in AND con il clock
- solo se Write = 1 il valore di Write data viene scritto in uno dei registri

Lettura da Register File

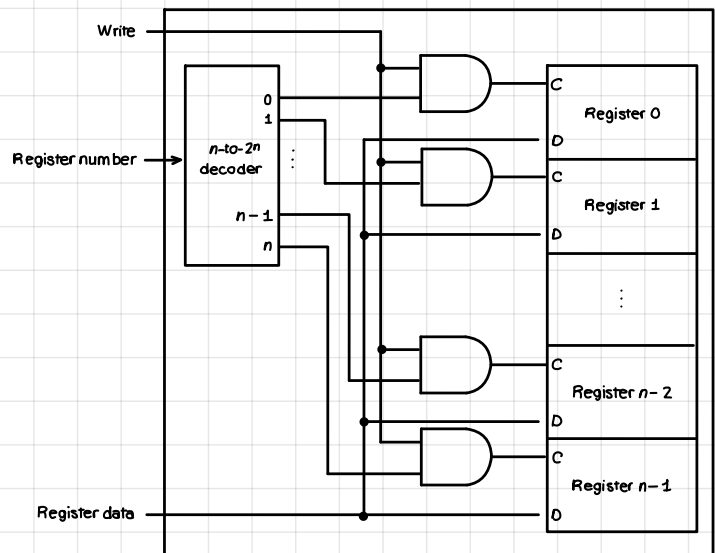
- Utilizza 2 segnali che indicano i registri da leggere (Read Reg 1, Read Reg 2)
- Utilizza 2 multiplexer: ognuno con 32 ingressi e un segnale di controllo
- Il register file fornisce sempre in output una coppia di registri



Scrittura nel Register file

Utilizza 3 segnali:

- Il registro da scrivere (Register Number)
 - Il valore da scrivere (Register Data)
 - Il segnale di controllo (Write)
-
- Utilizza un decoder che decodifica il numero del registro da scrivere (Write Register)
 - Il segnale Write (già in AND con il clock) è in AND con l'output del decoder
 - Se Write non è affermato nessun valore sarà scritto nel registro



Memoria

Oltre alle piccole memorie implementate per mezzo di registri e file di registro, esistono altri tipi di memorie che possiamo distinguere in base a diversi parametri:

1. Dimensione: quantità di dati memorizzabili
2. Velocità: l'intervallo di tempo tra la richiesta del dato e il momento in cui è disponibile
3. Consumo: potenza assorbita
4. Costo: costo per bit

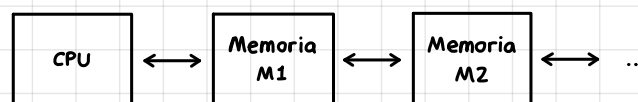
Idealmente un calcolatore dovrebbe avere quanta più memoria possibile, ad alta velocità, basso consumo e minimo costo.

Gerarchie di memoria

Non è possibile avere un'unica memoria con tutte le caratteristiche ideali

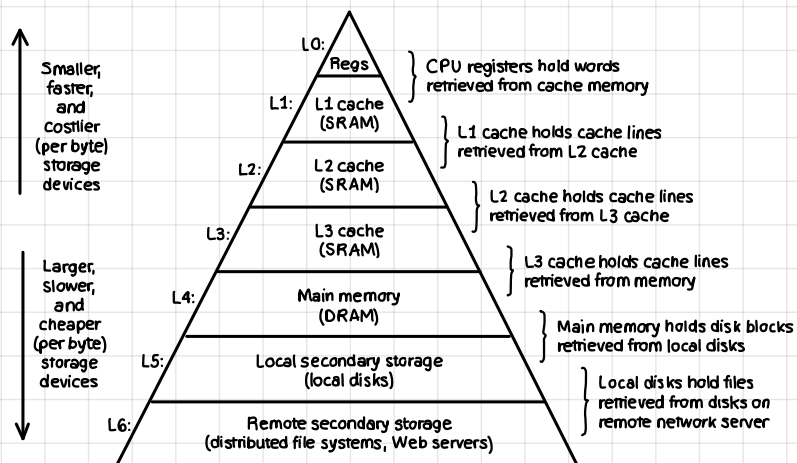
Possiamo però organizzare una sorta di gerarchia

- memorie piccole, più veloci (e costose) sono poste ai livelli alti
- memorie ampie, più lente (e meno costose) sono poste ai livelli più bassi



RAM (Random Access Memory)

- Meno veloce della memoria dei registri, ma molto più capiente
- E' detta RAM perche i tempi di accesso sono indipendenti dal valore dell'indirizzo della cella di memoria acceduta
- E' composta a piu livelli (gerarchie di memoria)
- E' un tipo di dato volatile (perde i dati se non alimentato)



SRAM (Static RAM)

La SRAM e piu veloce:

- Per la sua realizzazione vengono usati dei flip-flop (≈ 6 transistor per bit)
- E' usata per realizzare memorie veloci, come le memorie cache
- Tempi di accesso attorno al nanosecondo
- Dimensioni fino a qualche MB
- Nel passato erano in packages separati, oggi sono integrate nel processore (cache L1 e L2), in realta piu complesso, ci sono multicore, architetture NUMA

DRAM (Dynamic RAM)

La DRAM e piu capiente ma piu lenta:

- Tempi di accesso intorno al centinaio di nanosecondi
- Ogni bit e memorizzato tramite un condensatore (memoria piu densa della SRAM)
- I condensatori perdono la loro carica -> e necessario rinfrescare il contenuto delle DRAM a intervalli di tempo prefissati (circa 1ms)
- E' usata per realizzare memorie capienti e di accesso meno frequente (come la memoria principale)

SSRAM (Synchronous Static RAM)

La SSRAM è un tipo di memoria che va a leggere e scrivere nei registri di memoria solo quando è sincronizzata con il clock.

La più comune applicazione delle SSRAM è per le memorie Cache, utilizzate per fornire al processore le informazioni e i dati più frequentemente utilizzati.

SDRAM (Synchronous Dynamic RAM)

La SDRAM è stata sviluppata in risposta alla maggiore velocità di altri componenti dei computer. Precedentemente la memoria doveva essere asincrona, ovvero doveva operare indipendentemente dal processore.

La memoria sincrona sincronizza la risposta dei moduli di memoria con il bus di sistema.

Man mano che altri componenti del computer hanno accresciuto la propria velocità, era necessario adeguare anche la velocità della memoria

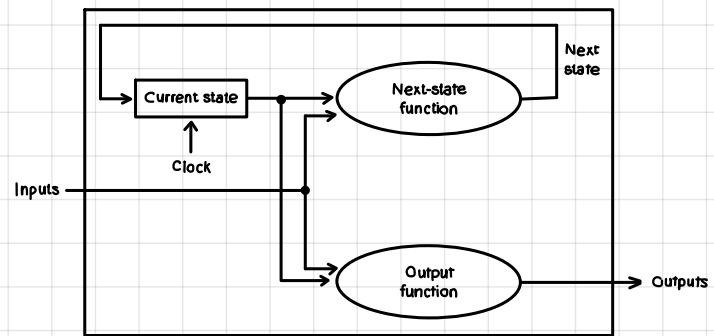
Macchine a stati finiti

Finite State Machine (FSM): usate per descrivere i circuiti sequenziali

Composte da un set di stati e 2 funzioni:

- Next state function – determina lo stato successivo partendo dallo stato corrente e dai valori in ingresso
- Output function – produce un insieme di risultati partendo dallo stato corrente e dai valori in ingresso

Sono sincronizzate con il clock



FSM - Moore vs Mealy

FSM – next – state dipende sia dagli input che dallo stato corrente

FSM – output:

- Se dipende solo dallo stato corrente: Moore – usato come controller
- Se dipende dallo stato corrente e dagli input: Mealy
- Moore and Mealy sono equivalenti e si possono trasformare autonomamente tra di loro
- In questo corso usiamo FSM Moore

